Developer Insights Bible — Full Current-State Export

Generated on 2026-03-25 01:31:09

Developer Insights Bible — The Attention Economy

Complete Technical Handover Document

Generated: 2026-03-25 00:31:04 UTC **Git branch:** `main` — commit `0f91ebb` **Repository root:** `/Users/sebbo/Desktop/Replit revival` **HuggingFace Space:** `https://sebbo89-earningscall2.hf.space/` **HF Space repo:** `https://huggingface.co/spaces/sebbo89/Earningscall2`

> This document is auto-generated by `scripts/generate_current_bible.py`. > Regenerate at any time with: `python3 scripts/refresh_dev_manual.py` > **Must be run after every change to `app/`, `scripts/`, or `.streamlit/`.**

Table of Contents

- 1. Project Overview
- 2. Architecture & Entry Points
- 3. Page Inventory
- 4. Home Page Render Order (Welcome.py)
- 5. Overview Page Architecture
- 6. Excel Workbook — Complete Sheet Inventory
- 7. Data Flow: Excel → Processors → Pages
- 8. Utils Module Catalog
- 9. Transcript Intelligence Pipeline
- 10. Environment Variables & Secrets
- 11. Deployment — HuggingFace Spaces
- 12. Function Inventory (Key Files)
- 13. Welcome.py Helper Functions (Active vs Legacy)
- 14. Chart & Component Footprint
- 15. Known Operational Risks
- 16. Runtime Diagnostics (Auto-Probed)
- 17. Recent Git History
- 18. Regeneration & Artifact Policy

1) Project Overview

The Attention Economy is a multi-page Streamlit dashboard that tracks 14 major media/tech companies (Alphabet, Amazon, Apple, Meta, Microsoft, Netflix, Disney, Comcast, Spotify, Roku, Warner Bros. Discovery, Paramount, Snap, Pinterest) across revenue, segments, subscribers, stock performance, and macro advertising market data.

Primary data source: A single Excel workbook (`Earnings + stocks copy.xlsx`) stored in `app/attached_assets/`. This file is ~17MB and tracked in git. The workbook contains 40 sheets covering financials, segments, macro data, country-level advertising, stock prices, and transcripts.

Secondary data source: Google Sheets (same workbook exported as XLSX). Used as fallback only when no valid local file is detected.

Intelligence layer: SQLite database (`earningscall_intelligence.db`) built from earnings call transcript .txt files via the intelligence pipeline. Powers the transcript quotes strip (Human Voice) and Genie AI chat.

2) Architecture & Entry Points

```
app.py                          ← Git root entrypoint
  ■■ sys.path.insert(0, 'app')  ← Makes app/ importable
  ■■ exec(app/Welcome.py)        ← Streamlit entry page

app/
  Welcome.py                    ← Home page (scrolling narrative)
  pages/
    00_Overview.py              ← Macro / global ad market
    01_Earnings.py              ← Company deep-dive
    02_Stocks.py                ← Stock charts
    03_Editorial.py             ← AI-generated insights
    04_Genie.py                 ← AI chat assistant
  utils/                        ← Shared utilities (~40 modules)
  attached_assets/              ← Excel workbook + logos
  data_processor.py             ← Main FinancialDataProcessor class
  .streamlit/config.toml        ← Streamlit config (no theme set)

scripts/                        ← Pipeline scripts (run outside Streamlit)
earningscall_transcripts/       ← Transcript .txt files + index CSVs
earningscall_intelligence.db    ← SQLite DB (gitignored, built at runtime)
reports/                        ← Auto-generated dev bible + assets
```

Theme: No Streamlit theme is set in config.toml — the app uses custom inline CSS to enforce a dark theme (`#0d1117` background) on the Home page. The Earnings and Overview pages use a semi-light theme via `utils/styles.py` with chart text tuned for dark background rendering.

3) Page Inventory

File	Lines	Description
`app/pages/00_Overview.py`	11513	Macro overview — global ad market, duopoly analysis, M2 money supply, concentration bars, landscape bubble chart, market bet. 8-section navigator with per-section st.stop() exits.
`app/pages/01_Earnings.py`	6208	Company deep-dive — hero banner, KPI cards, revenue waterfall, segment composition donut, segment evolution chart, transcript highlights, heatmap, AI chat (Genie).
`app/pages/02_Stocks.py`	1206	Stock price charts, returns, market cap trends, volume, daily/minute OHLCV data sourced from the 'Stocks & Crypto' sheet.
`app/pages/03_Editorial.py`	977	Editorial/insight narrative page. Renders auto-generated company and macro insights from the intelligence pipeline.
`app/pages/04_Genie.py`	3989	AI assistant (Genie) — natural-language Q&A about earnings data, powered by OpenAI API with SQLite transcript intelligence DB as context.

4) Home Page Render Order (Welcome.py)

1. Dark CSS + fonts injected (`st.markdown` with `<style>`)

2. Excel workbook resolved (`resolve_financial_data_xlsx`)
3. Intelligence pipeline freshness check (only if `AUTO_REFRESH...=1`)
4. Hero section: KPI strip (Global Ad Spend, Tracked Revenue, Market Cap) + narrative
5. `The World` — orthographic globe choropleth (ad spend % GDP per country)
6. `The Structural Shift` — animated donut (TV→Internet ad share 1999–2024)
7. `Attention + Duopoly` — animated scene (Google+Meta combined share)
8. `Concentration` — bitcoin-style horizontal bar showing duopoly + human sidebars
9. `Revenue Anatomy` — full-width flex bars per company (total + ad% breakdown)

10. `The World` — *Every dollar. Every country.* (`app/Welcome.py:2770`) 11. `THE STRUCTURAL SHIFT` — *Television had the world's total attention. Then new players came to compete.* (`app/Welcome.py:3038`) 12. `IF PLATFORMS WERE COUNTRIES` — *If the world were divided by platform, this is how it would look.* (`app/Welcome.py:3837`) 13. `ATTENTION ECONOMY` — *Who Owns Your Time* (`app/Welcome.py:3868`) 14. `THE CONCENTRATION` — *Most of it went to very few hands.* (`app/Welcome.py:3977`) 15. `REVENUE ANATOMY` — *Not all revenue is advertising.* (`app/Welcome.py:4181`) 16. `The Money Printer` — *When liquidity expands, ad markets follow.* (`app/Welcome.py:4335`) 17. `The Structural Shift` — *The ad market didn't just grow. It transformed.* (`app/Welcome.py:4387`) 18. `The Market Bet` — *Starting from the same base of 100, who compounded fastest?* (`app/Welcome.py:4457`) 19. `The Wealth Machine` — *Market cap, year by year.* (`app/Welcome.py:4546`) 20. `The Human Voice` — *Here's what management teams are saying.* (`app/Welcome.py:4915`) 21. Market tape (stock strip, `_render_stock_price_strip`) 22. Gateway section (→ Overview / Earnings / Genie)

5) Overview Page Architecture (00_Overview.py)

Navigator model: 8-section single-view with per-section `st.stop()` exits. Each section is self-contained — selecting a section renders only that section.

Key	Title	Description
`macro_geography`	Macro & Geography	Macro regime, liquidity context, and interactive global ad spend map.
`channels_devices`	Channels & Devices	Ad spend by channel, insights by category, and device-platform migration.
`deep_dives`	Company Dashboard	P/E, debt, margins, R&D, employees, and market structure comparison.
`narrative_sentiment`	Narrative & Sentiment	Transcript topics, CEO/CFO quotes, and market signal cards.

6) Excel Workbook — Complete Sheet Inventory

File: `app/attached_assets/Earnings + stocks copy.xlsx` (~17MB, git-tracked)

This single file is the primary data source for all pages. The `resolve_financial_data_xlsx()` function in `utils/workbook_source.py` auto-detects it and validates: ≥43 sheets, required tabs present, ≥5 companies, ≥8 years, earliest year ≤ 2015.

Sheet Catalog (Purpose & Usage)

Sheet Name	Purpose	Key Columns	Used By
`Stocks & Crypto`	Daily OHLCV price + market cap for all tracked tickers	date, price, open, high, low, vol., change%, market cap	02_Stocks.py, Welcome.py (market tape)

Sheet Name	Purpose	Key Columns	Used By
`USD Inflation`	Historical US CPI / inflation rates (official + shadow)	Year, Official Headline CPI, Core Inflation, ShadowStats*	00_Overview.py (macro panel)
`Nasdaq Composite Est. (FRED)`	Nasdaq index + estimated market cap time-series (FRED)	observation_date, NASDAQCOM, estimated_nasdaq_market_cap_usd	00_Overview.py
`Country_Totals_vs_GDP`	Ad spend per country vs GDP (yearly, country-level)	Country, Year, AdSpending_Total, AdSpending_USD, GDP_USD, Ad_vs_GDP_%	Welcome.py (globe choropleth)
`Country_Advertising_Data_FullVi`	Full ad spend breakdown by country/ year/metric_type	Country, Year, Metric_type, Value	00_Overview.py
`Country_avg_timespent_intrnt24`	Average daily internet time per country (2024)	Country, Daily Time Spent Internet (hours.minutes)	00_Overview.py
`Global_Adv_Aggregates`	Global ad market totals by year and metric_type	year, metric_type, value	Welcome.py, 00_Overview.py
`Global Advertising (GroupM)`	GroupM annual global ad spend by format (TV, Search, etc.)	Year, Traditional_TV, Connected_TV, Search, NonSearch, Retail_Media	Welcome.py hero KPI, 00_Overview.py
` (GroupM) Granular `	GroupM granular ad spend with more format detail	Year, TV/Pro Video, Audio, Newspapers, Internet, ...	00_Overview.py
`M2_values`	US M2 money supply (legacy sheet name; new sheet is 'M2')	observation_date / USD observation_date, WM2NS / M2SL	00_Overview.py, 01_Earnings.py, utils/m2_supply_data.py
`Company_metrics_earnings_values`	Annual financial metrics per company (the main data table)	Company, Year, Revenue, Operating Income, Net Income, Debt, R&D, CapEx, EPS, FCF, EBITDA, Margin	01_Earnings.py, 00_Overview.py, data_processor.py
`Company_Employees`	Employee headcount per company per year	Company, Year, Employee Count	01_Earnings.py (pie chart center label)
`Company_Segments_insights_text`	AI-generated segment-level insights per company/year/quarter	Company, Year, Segment, Insight, Category	01_Earnings.py (segment insights panel)
`Company_insights_text`	AI-generated company-level insights (annual + quarterly)	company, year, category, insight, quarter	01_Earnings.py (insights panel)
`Company_advertising_revenue`	Advertising revenue per company per year	Year, Google_Ads, Meta_Ads, Amazon_Ads, Spotify_Ads, ...	Welcome.py (revenue anatomy), 00_Overview.py
`Company_subscribers_values`	Subscriber counts per service/quarter/year	service, quarter, year, subscribers, unit, US_Canada, International	01_Earnings.py

Sheet Name	Purpose	Key Columns	Used By
`Hardware_Smartphone_Shipments`	Global smartphone shipment data by OEM and year	Year, Apple_iPhone_Units_M, Samsung_Units_M, ...	00_Overview.py
`Macro_Wealth_by_Generation`	Generational wealth distribution data by country/year	Country, Year, Age_Group, Generation_Label, Total_Wealth_Billion_USD, Wealth_Share_Pct	00_Overview.py
`Company_revenue_by_region`	Revenue split by geographic region per company/year	company, year, segment_name, revenue_millions	01_Earnings.py
`Company_minute&dollar_earned`	Revenue-per-minute-watched and engagement metrics	Platform, Subscribers/Users, Avg Time per User, Total Minutes, Revenue, $ per Minute	Welcome.py (dependency chart / landscape)
`Company_yearly_segments_values`	Annual segment revenue per company	Company, year, segments, Yearly Segment Revenue	01_Earnings.py (segment composition + evolution)
`Company_Quarterly_segments_valu`	Quarterly financial metrics per company (income statement)	Ticker, Year, Revenue, Cost Of Revenue, Operating Income, Net Income, Capex, R&D, ...	01_Earnings.py
`Alphabet Quarterly Segments`	Alphabet segment revenue by quarter	Quarter, Search & Other, YouTube Ads, Google Network, Google Cloud, ...	01_Earnings.py
`Apple Quarterly Segments`	Apple segment revenue by quarter	Quarter, iPhone, Mac, iPad, Wearables, Services	01_Earnings.py
`Amazon Quarterly Segments`	Amazon segment revenue by quarter	Quarter, Online Stores, 3P Seller, AWS, Advertising, ...	01_Earnings.py
`Meta Quarterly Segments`	Meta segment revenue by quarter	Quarter, Family of Apps, Reality Labs	01_Earnings.py
`Comcast Quarterly Segments Gran`	Comcast granular quarterly segment breakdown	Quarter, Domestic Broadband, Wireless, Video, Advertising, ...	01_Earnings.py
`Disney Quarterly Segments`	Disney segment revenue by quarter	Quarter, ...	01_Earnings.py
`Microsoft Quarterly Segments`	Microsoft segment revenue by quarter	Quarter, Cloud, Office, Windows, Gaming, LinkedIn, Search, ...	01_Earnings.py
`Netflix Quarterly Segments`	Netflix revenue by region per quarter	Quarter, UCAN, EMEA, LATAM, APAC	01_Earnings.py
`Paramount Quarterly Segments`	Paramount segment revenue by quarter	Quarter, DTC, TV Media, Filmed Entertainment	01_Earnings.py
`Roku Quarterly Segments`	Roku segment revenue by quarter	Quarter, Platform, Devices/Player	01_Earnings.py
`Spotify Quarterly Segments`	Spotify segment revenue by quarter	Quarter, Premium, Ad-supported	01_Earnings.py

Sheet Name	Purpose	Key Columns	Used By
`Warner Bros Quarterly Segments`	WBD segment revenue by quarter	Quarter, Distribution, Advertising, Content, Other	01_Earnings.py
`Overview_Macro`	Curated macro snapshot row for Overview dashboard	year, quarter, m2_value, global_ad_market, duopoly_share, ...	00_Overview.py
`Overview_Insights`	Manually curated insight rows for Overview page	insight_id, sort_order, category, title, year, quarter, comment	00_Overview.py
`Overview_Charts`	Chart metadata (titles, comments) for Overview charts	chart_key, year, quarter, title, pre_comment, post_comment	00_Overview.py
`Transcripts`	Raw earnings call transcript text per company/year/quarter	company, year, quarter, transcript_text, last_updated	scripts/rebuild_transcript_index.py, intelligence pipeline
`Overview_Auto_Insights`	Auto-generated macro insights for Overview page	insight_id, category, title, text, comment, priority, companies	00_Overview.py

Live Sheet Dimensions (Probed at Generation Time)

Sheet Name	Dimensions	First Columns (up to 6)
(probe failed)	No module named 'requests'	

7) Data Flow: Excel → Processors → Pages

```
Excel Workbook (app/attached_assets/*.xlsx)
  ■
  ■■ resolve_financial_data_xlsx()        ← utils/workbook_source.py
  ■    Resolution order:
  ■    1. Explicit local candidates passed by caller
  ■    2. Auto-detect *.xlsx in app/attached_assets/
  ■    3. Download from GOOGLE_SHEET_URL (20s timeout)
  ■
  ■■ FinancialDataProcessor               ← app/data_processor.py
  ■    .load_data()  →  reads sheets:
  ■      Company_metrics_earnings_values  → df_metrics
  ■      Company_yearly_segments_values   → df_segments
  ■      Company_subscribers_values       → df_subscribers
  ■      Company_Employees                → employee data
  ■      Company_revenue_by_region        → regional data
  ■    Cached via get_data_processor() in utils/state_management.py
  ■
  ■■ _read_excel_sheet_cached()           ← Welcome.py internal
  ■    Reads individual sheets on demand (KPIs, map, anatomy, etc.)
  ■    Cached with @st.cache_data(ttl=4h)
  ■
  ■■ load_quarterly_segments()            ← 01_Earnings.py internal
  ■    Reads per-company 'X Quarterly Segments' sheets
  ■
  ■■ load_combined_stock_market_data()    ← utils/workbook_market_data.py
       Reads 'Stocks & Crypto' sheet
       → Used by Welcome.py (market tape) and 02_Stocks.py

SQLite (earningscall_intelligence.db)
  Built by: scripts/build_intelligence_db.py
  Tables: transcript_highlights, transcript_topics, topic_metrics
  Used by: Welcome.py (Human Voice strip), 04_Genie.py (Genie AI context)
```

8) Utils Module Catalog (`app/utils/`)

Module	Exists	Lines	Description
`workbook_source.py`	✓	304	Resolves the primary Excel data file — tries local files first (attached_assets/*.xlsx), then falls back to Google Sheets download with 20s timeout. Validates sheet count, required tabs, and financial coverage.
`workbook_market_data.py`	✓	331	Loads stock/market data from the 'Stocks & Crypto' sheet. Builds company-ticker maps for market tape and 02_Stocks.py.
`state_management.py`	✓	151	Provides get_data_processor() — a cached singleton FinancialDataProcessor instance shared across all pages via Streamlit session state.
`data_granularity.py`	✓	256	Detects available data granularity (annual/quarterly/monthly) per company from the workbook sheets.
`data_availability.py`	✓	143	Returns available quarters for a given company/year pair from quarterly segment sheets.
`m2_supply_data.py`	✓	471	Loads M2 money supply data. Supports both old sheet ('M2_values', columns WM2NS/USD observation_date) and new sheet ('M2', columns M2SL/observation_date) with automatic fallback.
`data_loader.py`	✓	399	Generic data loading helpers used by data_processor.py.
`auth.py`	✓	18	Password gate (check_password()). Called on pages that require authentication.
`header.py`	✓	311	Renders the top navigation header (display_header()). Shared across all pages.

Module	Exists	Lines	Description
`logos.py`	✓	72	Loads company logos from attached_assets/ as base64. Used in market tape, Earnings hero, pulse strip.
`global_fonts.py`	✓	119	Injects Google Fonts (DM Sans, Syne, Montserrat) via st.markdown. Called at top of every page.
`styles.py`	✓	1370	CSS + Plotly theme definitions. get_page_style() returns the light-theme CSS used by Earnings/Overview. apply_plotly_theme() sets the default Plotly template ('mfe_blue' based on plotly_white).
`theme.py`	✓	927	get_theme_mode() — returns current light/dark preference.
`components.py`	✓	236	render_ai_assistant() — renders the Genie AI chat UI component.
`transcript_startup_sync.py`	✓	194	One-time-per-container sync of local transcript text files into the Excel workbook's 'Transcripts' sheet.
`live_stock_feed.py`	✓	124	Fetches live or cached stock prices for the market tape strip on Welcome.py.
`macro_trends.py`	✓	332	Macro data helpers (Fed Funds rate, inflation, M2 trend calculations).
`fed_funds_data.py`	✓	155	Loads Federal Funds Rate data for macro panel.
`inflation_analysis.py`	✓	368	Inflation analysis helpers — CPI comparisons, real vs nominal calculations.
`insights.py`	✓	328	Generates/retrieves insight text for display on Overview and Earnings pages.
`insights_loader.py`	✓	254	Loads generated_insights_latest.csv for display.
`database_service.py`	✓	181	SQLite service wrapper for earningscall_intelligence.db.
`api_client.py`	✓	195	API client for external data sources.

Module	Exists	Lines	Description
`genie_ai.py`	✓	471	Core Genie AI logic — builds context from transcript DB + sends to OpenAI.
`openai_service.py`	✓	612	OpenAI API wrapper used by Genie.
`ai_assistant.py`	✓	340	AI assistant UI helpers.
`page_transition.py`	✓	71	Page transition animation helpers.
`helpers.py`	✓	86	General utility functions shared across pages.

Additional utils (not yet catalogued):

- `ai_chat.py` (933 lines)
- `ai_service.py` (129 lines)
- `animation_helper.py` (382 lines)
- `anthropic_service.py` (97 lines)
- `api_key_manager.py` (136 lines)
- `bar_animation_styles.py` (350 lines)
- `bitcoin_analysis.py` (316 lines)
- `csv_data_loader.py` (214 lines)
- `enhanced_ai_chat.py` (405 lines)
- `enhanced_chat_interface.py` (401 lines)
- `inflation_calculator.py` (457 lines)
- `insights_loader_fixed.py` (254 lines)
- `language.py` (368 lines)
- `optimized_data_loader.py` (136 lines)
- `sql_assistant_sidebar.py` (534 lines)
- `thought_map.py` (1906 lines)
- `time_utils.py` (74 lines)
- `transcript_live.py` (705 lines)
- `user_role.py` (289 lines)
- `workflow_manager.py` (57 lines)

9) Transcript Intelligence Pipeline

Orchestrator: `scripts/sync_all_intelligence.py`

Input: Earnings call transcripts as `.txt` files in `earningscall_transcripts/` **Output:** SQLite DB + CSV files consumed by Home page and Genie AI

- `run_step("1/7 Rebuild transcript index", [py, "scripts/rebuild_transcript_index.py", "--root", args.root], repo_root)`
- `run_step("2/7 Extract topics + metrics", [py, "scripts/extract_transcript_topics.py", "--root", args.root], repo_root)`
- `run_step("3/7 Extract KPI values", [py, "scripts/extract_kpi_values.py", "--root", args.root], repo_root)`

- `run_step("4/7 Extract transcript highlights", [py, "scripts/extract_transcript_highlights_from_sheet.py", "--out-dir", args.root], repo_root)`
- `run_step("5/7 Build intelligence DB", [py, "scripts/build_intelligence_db.py", "--db", args.db, "--root", args.root], repo_root)`
- `run_step("6/7 Sync workbook metrics to SQL", [py, "scripts/sync_gsheet_to_sql.py", "--db", args.db], repo_root)`
- `run_step("7/7 Generate summary insights", [py, "scripts/generate_insights.py", "--db", args.db], repo_root)`

Transcript CSVs in `earningscall_transcripts/`:

- `transcript_index.csv` (present): Master index: company/year/quarter/file path
- `transcript_topics.csv` (present): Topics extracted per transcript
- `transcript_kpis.csv` (present): KPI values mentioned in transcripts
- `transcript_highlights.csv` (present): Key quote highlights per transcript
- `overview_iconic_quotes.csv` (present): Hand-picked / top-ranked quotes for Overview
- `topic_metrics.csv` (present): Aggregated topic frequency metrics
- `generated_insights_latest.csv` (present): AI-generated insight text (latest run)

Trigger: Pipeline only runs automatically if `AUTO_REFRESH_INTELLIGENCE_PIPELINE_ON_STARTUP=1`. Otherwise run manually: `python3 scripts/sync_all_intelligence.py`

10) Environment Variables & Secrets

Variable	Required?	Description
GOOGLE_SHEET_URL	Required	URL to the Google Sheet XLSX export. Used by workbook_source.py as fallback if no local XLSX found.
AUTO_REFRESH_INTELLIGENCE_PIPELINE_ON_STARTUP	Optional	Set to '1'/'true' to run the transcript intelligence pipeline on app startup. Disabled by default on HF to keep cold start fast.
AUTO_SYNC_TRANSCRIPTS_ON_STARTUP	Optional	Set to '1'/'true' to sync local transcript .txt files into the Excel workbook on startup. Disabled by default.
OPENAI_API_KEY	Required for Genie	OpenAI API key for the Genie AI assistant (04_Genie.py).
HF_TOKEN	Deployment	HuggingFace token for pushing to the Space. Used in git remote: https://user:HF_TOKEN@huggingface.co/spaces/sebbo89/Earningscall2

Where to set on HuggingFace: Space → Settings → Repository secrets

11) Deployment — HuggingFace Spaces

Platform: HuggingFace Spaces (free tier, Streamlit runtime) **Live URL:** `https://sebbo89-earningscall2.hf.space/` **Space repo:** `https://huggingface.co/spaces/sebbo89/Earningscall2`

Git remote setup:

```
git remote add hf https://huggingface.co/spaces/sebbo89/Earningscall2
git push hf main
```

Push workflow (from CODEX_RULES.md):

```
# 1. Compile-check all 3 pages
PYTHONPYCACHEPREFIX=/tmp/pycache python3 -m py_compile app/Welcome.py
PYTHONPYCACHEPREFIX=/tmp/pycache python3 -m py_compile app/pages/00_Overview.py
```

```
PYTHONPYCACHEPREFIX=/tmp/pycache python3 -m py_compile app/pages/01_Earnings.py
# 2. Regenerate dev bible
python3 scripts/refresh_dev_manual.py
# 3. Push to HuggingFace
git push hf main
```

Cold start behaviour: Free HF tier sleeps containers after inactivity. Container wake takes ~30–60s (unavoidable). Data loading is ~2s (local XLSX).

Large files — NEVER commit:

- `earningscall_intelligence.db` (gitignored)
- `app/attached_assets/HeroVideo.mp4` (gitignored)
- Any file over 5MB (except the tracked XLSX which is ~17MB)

12) Function Inventory (Key Files)

File	Lines	Functions	Classes
`app.py`	5	0	0
`app/Welcome.py`	5003	67	0
`app/pages/00_Overview.py`	11513	150	0
`app/pages/01_Earnings.py`	6208	1	0
`app/pages/02_Stocks.py`	1206	20	0
`app/pages/03_Editorial.py`	977	7	0
`app/pages/04_Genie.py`	3989	30	0
`app/data_processor.py`	1005	1	1
`app/stock_processor_fix.py`	254	5	1
`app/utils/workbook_source.py`	304	12	0
`app/utils/workbook_market_data.py`	331	12	0
`app/utils/transcript_startup_sync.py`	194	9	1
`scripts/rebuild_transcript_index.py`	199	7	1
`scripts/extract_transcript_topics.py`	682	11	0
`scripts/extract_transcript_highlights_from_sheet.py`	500	20	0
`scripts/extract_kpi_values.py`	38	1	0
`scripts/build_intelligence_db.py`	349	11	0
`scripts/sync_gsheet_to_sql.py`	435	13	0
`scripts/generate_insights.py`	808	5	1
`scripts/sync_all_intelligence.py`	42	2	0

13) Welcome.py Helper Functions (Active vs Legacy)

Active (called at least once in file):

Function	Line	Internal Calls	Name References
`ensure_intelligence_pipeline_is_fresh`	28	1	2

Function	Line	Internal Calls	Name References
`_run_startup_transcript_sync`	252	1	2
`_mount_welcome_liquid_background`	305	1	2
`_format_money_musd`	452	5	6
`_company_color`	502	4	5
`_resolve_logo`	541	10	11
`_resolve_workbook_path`	680	1	2
`_normalize_quarter_label`	736	4	9
`_normalize_company_name`	752	6	12
`_company_variants`	759	2	3
`_resolve_speaker_label`	780	2	3
`_split_company_tokens`	800	1	2
`_read_excel_sheet_cached`	811	12	13
`_pick_col`	826	7	8
`_load_company_metrics_sheet`	837	2	3
`_load_overview_macro_sheet`	876	1	2
`_load_company_ad_revenue_sheet`	931	3	4
`_load_company_employees_sheet`	949	1	2
`_select_latest_quarter_for_year`	978	1	2
`_load_m2_yearly_series`	991	1	2
`_map_ad_column_to_company`	1169	1	2
`_load_ad_revenue_by_company`	1179	2	3
`_format_people`	1204	1	2
`_sqlite_has_column`	1318	2	3
`_normalize_quotes_frame`	1424	2	3
`_load_pulse_quotes_csv`	1496	1	2
`_load_pulse_quotes_sqlite`	1516	1	2
`_load_transcript_pulse_quotes`	1569	1	2
`_load_page_data`	1612	1	2
`_section`	2269	11	12
`_separator`	2287	13	14
`_deep_dive`	2294	9	10
`_find_col`	2321	35	36
`_yr`	2334	1	11
`_yoy`	2347	3	4

Function	Line	Internal Calls	Name References
`_yoy_vec`	2353	1	2
`_apply_dark_chart_layout`	2357	3	4
`_normalize_market_feed`	2383	1	2
`_load_market_feed`	2408	1	2
`_render_transcript_pulse_strip`	2433	1	2
`_render_stock_price_strip`	2495	1	2
`_build_numeric_iso_map`	2732	2	3
`_build_ss_html`	2954	1	2
`_build_attn_html`	3045	1	2
`_safe_float`	3238	10	11
`_load_platform_subscriber_data`	3414	1	2
`_build_timeline_data`	3473	1	2

Possibly legacy (call count = 0 within Welcome.py):

Function	Line	Internal Calls	Name References
`_to_numeric`	448	0	1
`_safe_pct`	469	0	1
`_build_hero_narrative`	477	0	1
`_render_company_logos`	558	0	1
`_clean_signal_title`	579	0	1
`_build_hero_company_logo_bar`	585	0	1
`_render_leaderboard_strip`	620	0	1
`_quarter_sort_value`	748	0	5
`_normalize_text_for_compare`	805	0	1
`_load_auto_insights`	899	0	1
`_build_home_narrative`	1052	0	1
`_macro_comment_is_placeholder`	1140	0	1
`_pick_macro_comment_for_period`	1147	0	1
`_build_bubble_dataset`	1216	0	1
`_pick_primary_company_for_insight`	1296	0	1
`_get_best_quote_for_insight`	1327	0	1
`_build_rank_cards`	1873	0	1
`_parse_human_count_millions`	3247	0	2
`_parse_billions`	3261	0	2
`_parse_trillion_minutes`	3276	0	2

14) Chart & Component Footprint

File	Plotly Calls	HTML Components	@st.cache_data	st.stop Calls
`app/Welcome.py`	4	14	17	0
`app/pages/00_Overview.py`	61	0	41	6
`app/pages/01_Earnings.py`	2	0	18	4
`app/pages/04_Genie.py`	3	0	18	1

15) Known Operational Risks

- **CSS bleeding (global selectors):** Welcome.py injects CSS using `.stApp`, `body`, `.main` — these apply globally. Earnings.py charts now use `render_plotly(light_theme=False)` by default (dark theme). If Earnings text appears invisible, check `render_plotly()` in 01_Earnings.py.
- **M2 sheet dual naming:** Old workbooks have sheet `M2_values` with columns `USD observation_date`/`WM2NS`. New workbooks use sheet `M2` with columns `observation_date`/`M2SL`. All 4 load points now try both names via try-loop.
- **Overview complexity:** `00_Overview.py` is >10k lines with many branch `st.stop()` exits — hard to reason, easy to regress.
- **HF cold start latency:** Free HF tier: ~30–60s container wake (unavoidable). Data load itself is ~2s via local XLSX.
- **SQLite DB not in git:** `earningscall_intelligence.db` is gitignored. Genie AI and Human Voice strip fail silently if DB missing.
- **No auto-regeneration of bible:** Bible does NOT auto-regenerate. Must run `python3 scripts/refresh_dev_manual.py` manually after every change to app/, scripts/, or .streamlit/.
- **Unused utility drift:** `app/utils/` has ~40 modules; several may be legacy. `insights_loader_fixed.py`, `optimized_data_loader.py` look like duplicates.

16) Runtime Diagnostics (Auto-Probed at Generation Time)

Check	Status	Time	Details
`Workbook resolver`	error	0.10s	ModuleNotFoundError: No module named 'pandas'
`FinancialDataProcessor.load_data()`	error	0.04s	ModuleNotFoundError: No module named 'pandas'
`SQLite intelligence tables`	ok	0.05s	db_found=False

- Slowest successful probe: **SQLite intelligence tables** at `0.05s`
- Checks needing attention:
- `Workbook resolver` → `error` (ModuleNotFoundError: No module named 'pandas')
- `FinancialDataProcessor.load_data()` → `error` (ModuleNotFoundError: No module named 'pandas')

17) Recent Git History (Last 15 Commits)

SHA	Date	Commit Message
0f91ebb	2026-03-25	Fix variable ordering after Earnings section reorder

SHA	Date	Commit Message
a94fb40	2026-03-25	Fix expander arrow text, pill clipping, reorder Earnings sections
e37b589	2026-03-24	Fix invisible expander labels, pill clipping, duplicate map panels, elaborate btn
00bf686	2026-03-24	Fix expander arrows, queued text, pill clipping, segments, Start Genie button
713575c	2026-03-24	Improve thought map: full map on load, visible edges, Clear All, Elaborate, Start Genie
fd0d2da	2026-03-24	Fix Overview axis colors, year selector, macro KPIs, plotly duplicate keys; fix Earnings Comcast segments, expander icons, hero blur
717a49d	2026-03-24	Improve Genie thought map: view buttons inside map, merge Fit/Refresh, add New Thought + node removal
551d879	2026-03-24	Fix Overview data loading, Stocks grid gaps, Earnings UI issues
17ce512	2026-03-24	Fix Stocks buttons and grid gaps (root cause found)
e0d5e48	2026-03-24	Fix Stocks buttons, Genie formatting, Overview restructure
0f3ca51	2026-03-23	Add Genie thought map view modes
1985aef	2026-03-23	Polish Genie thought map note flow
a6be452	2026-03-23	Fix Genie chat placeholder and thought map notes
f61c9ff	2026-03-23	fix: merge duplicate title keys in Plotly axis configs
1041a1d	2026-03-23	fix: Plotly titlefont ValueError, Stocks button JS injection

18) Regeneration & Artifact Policy

To regenerate this bible after any code change:

```
python3 scripts/refresh_dev_manual.py
```

This runs:

1. `scripts/generate_dev_manual_assets.py` — PNG charts
2. `scripts/generate_current_bible.py` — this markdown file
3. `scripts/generate_current_bible_pdf.py` — PDF export
4. Removes legacy duplicate files

5. Writes `reports/DEV_MANUAL_STATUS.json`

Canonical outputs (never recreate old filenames):

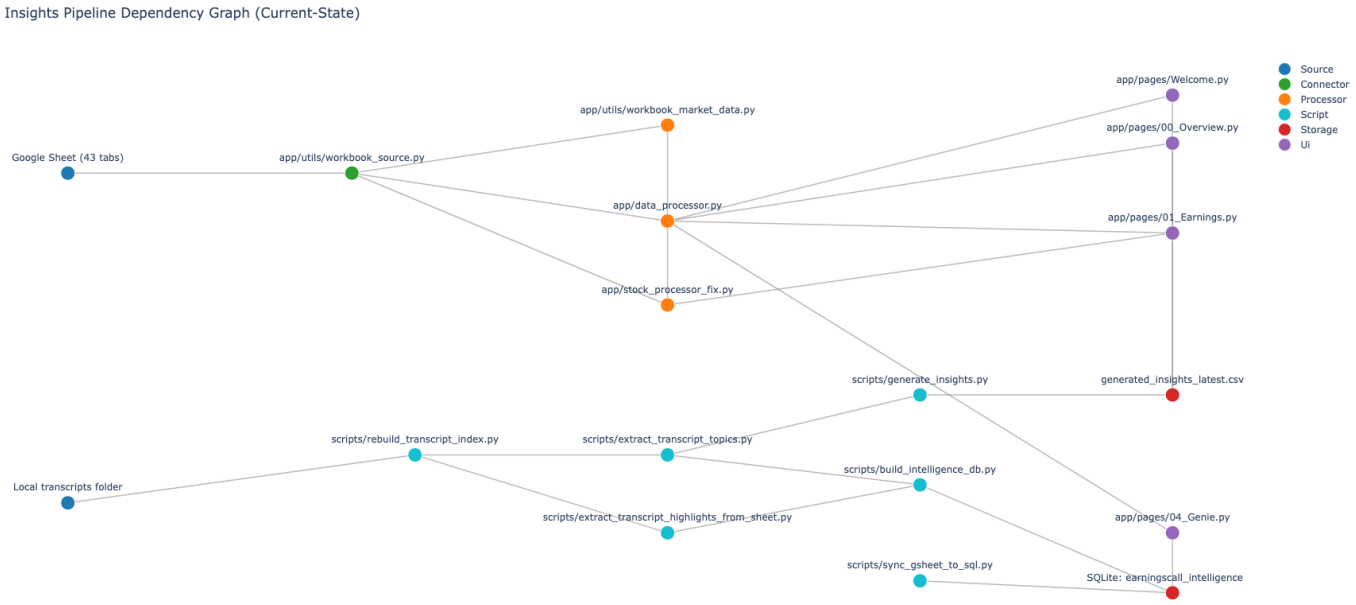
- `reports/Developer_Insights_Bible_CURRENT.md`
- `reports/Developer_Insights_Bible_CURRENT_Full.pdf`

Legacy files that must NOT be recreated:

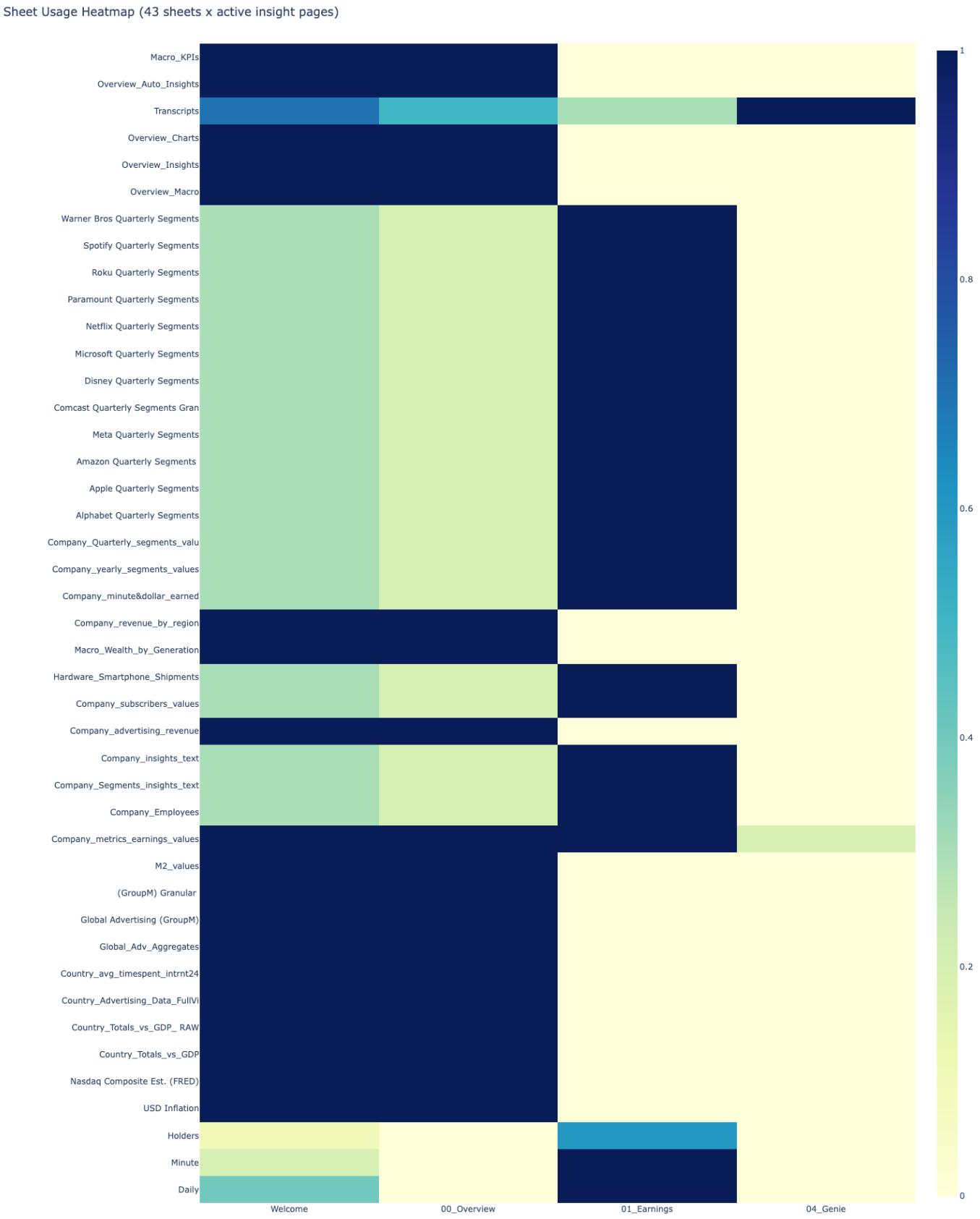
- `reports/Developer_Insights_Bible.md`
- `reports/Developer_Insights_Bible_Full.pdf`

Appendix — Visual Assets

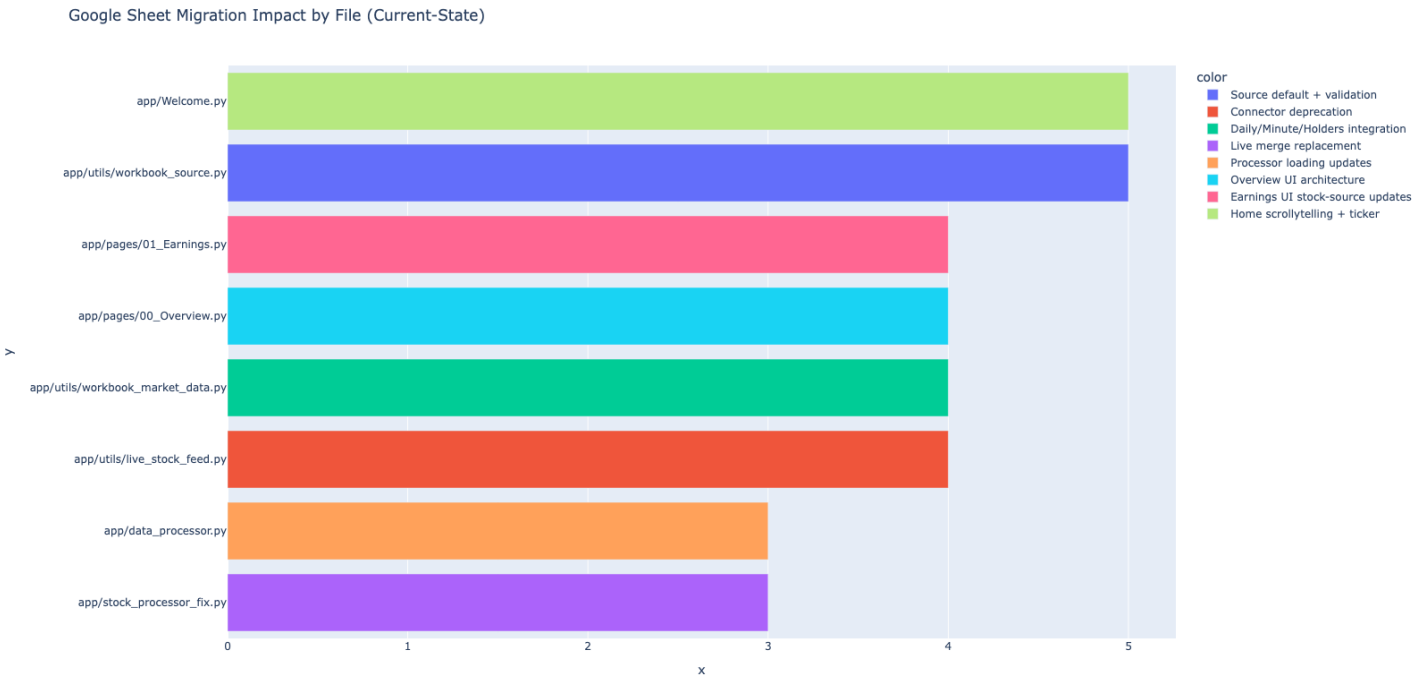
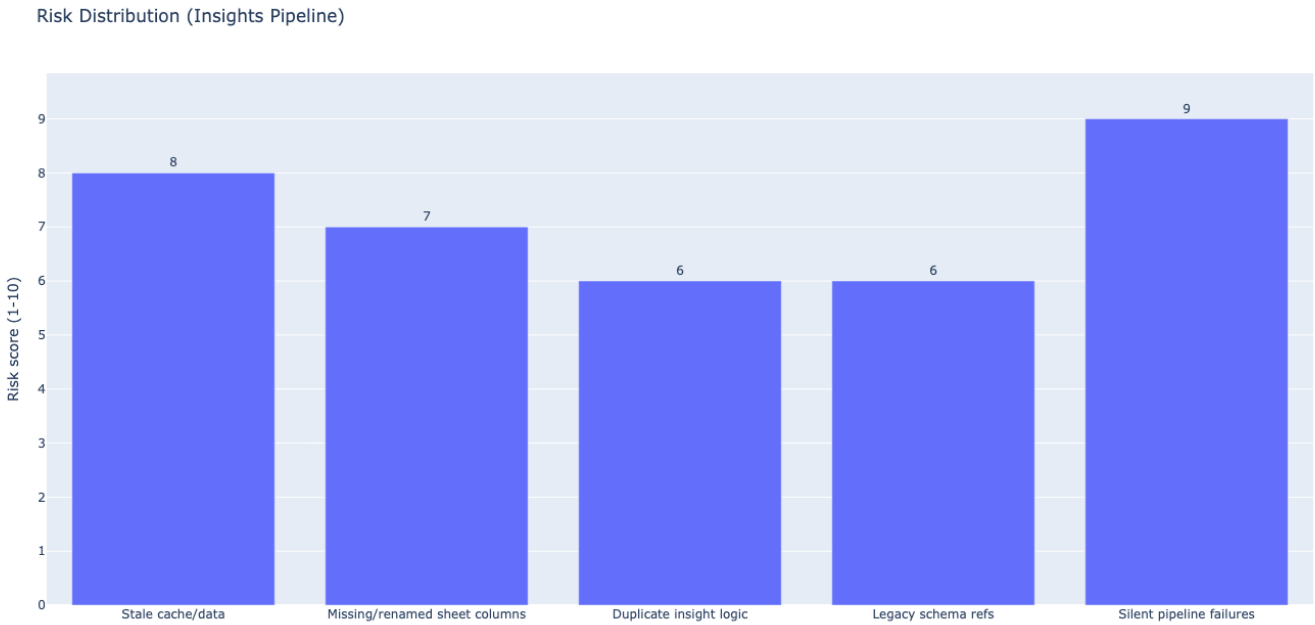
pipeline_dependency_graph.png



sheet_usage_heatmap.png



migration_impact.png



risk_distribution.png